

Exerciții C# — Liste în interiorul claselor

Nivel începător+ · o clasă care *deține* o listă și are metode peste ea (tipar manager/catalog)

mycodeschool

Pentru: Ana

Ce e nou: un câmp poate fi o *listă*

Până acum ai ținut lista în Main:

```
List<Produs> produse = new List<Produs>();  
produse.Add(...);
```

Ideea nouă: o clasă poate avea o listă drept câmp, iar metodele ei lucrează peste acea listă. Așa arată aplicațiile reale — un obiect „manager” care ține datele și știe să opereze pe ele.

```
internal class Cos  
{  
    public List<Produs> produse = new List<Produs>();    // lista trăiește în obiect  
  
    public void Adauga(Produs p)        // metodă care schimbă lista  
    {  
        this.produse.Add(p);  
    }  
  
    public double Total()              // metodă care parcurge lista și calculează  
    {  
        double suma = 0;  
        for (int i = 0; i < this.produse.Count; i++)  
        {  
            suma += this.produse[i].pret;  
        }  
        return suma;  
    }  
}
```

În Main devine curat de tot:

```
Cos cos = new Cos();  
cos.Adauga(unProdus);  
Console.WriteLine(cos.Total());
```

Tiparul din tot materialul: parcurgerea cu `for + .Count` (o știi deja) se mută **din Main în metodă**. `this.produse` e lista obiectului curent.

Partea 1 — Clasa Cos (coş de cumpărături)

Exerciţiul 1 — Cos cu Adauga şi NumaraProduse

Creează clasa `Cos` cu un câmp `List<Produs> produse`. Adaugă metodele:

- `Adauga(Produs p)` — pune produsul în listă;
- `NumaraProduse()` — întoarce câte produse sunt (`return this.produse.Count;`).

În `Main`: creezi un `Cos`, adaugi 3 produse, afișezi câte sunt.

Exemplu: adaugi 3 produse → `NumaraProduse()` întoarce 3.

Nu uita = `new List<Produs>();` la câmp, altfel lista e `null` și `Add` crapă.

Exercițiul 2 — Total din coș

Adaugă metoda `Total()` care parcurge lista din coș și întoarce **suma prețurilor**.

Exemplu: produse de 5, 10, 12.5 $\rightarrow$ `Total()` = 27.5.

E fix acumulatorul de la exercițiile cu liste, dar mutat în metodă și lucrând pe `this.produce`.

Exercițiul 3 — ValoareTotala (pret × stoc)

Adaugă `ValoareTotala()` care întoarce suma lui `pret * stoc` pentru toate produsele (folosește metoda `ValoareStoc()` pe fiecare produs).

Exemplu: $(12.5 \times 2) + (10 \times 1) + (5 \times 10) = 85$.

O metodă a coșului cheamă o metodă a produsului: `this.produse[i].ValoareStoc()`. Așa se „vorbesc” obiectele între ele.

Exercițiul 4 — AfiseazaTot

Adaugă `AfiseazaTot()` (metodă `void`) care parcurge lista și afișează `Descriere()` pentru fiecare produs. În `Main` apelezi doar `cos.AfiseazaTot();`.

Exemplu: afișează descrierea fiecărui produs din coș, pe rând.

Observă cât de curat devine `Main`: o singură linie în loc de un `for` întreg. Asta e ideea — complexitatea stă în clasă.

Partea 2 — Căutare și filtrare în listă

Exercițiul 5 — Cauta după nume (întoarce obiectul sau null)

Adaugă `Cauta(string nume)` care parcurge lista și **întoarce produsul** cu acel nume. Dacă nu-l găsește, întoarce `null`. În `Main` verifici rezultatul.

Exemplu: `Cauta("lanterna")` → produsul respectiv. `Cauta("xyz")` → `null` → afișezi „Nu există în coș”.

Tipul de retur al metodei devine `Produs`. După apel: `if (gasit != null) { ... }`.

Exercițiul 6 — Contine (întoarce bool)

Adaugă `Contine(string nume)` care întoarce `true` dacă există în coș un produs cu acel nume, altfel `false`.

Exemplu: `Contine("bec")` → `true` sau `false`.

Poți parcurge și da `return true;` de îndată ce-l găsești; `return false;` la final. SAU refolosește
ex. 5: `return Cauta(nume) != null;`.

Exercițiul 7 — CelMaiScump din coș

Adaugă `CelMaiScump()` care întoarce **produsul** cu prețul maxim din listă.

Exemplu: produse surub 12.5, bec 10, baterie 5 → întoarce surub.

Tiparul de maxim, dar ținem tot **obiectul**, nu doar prețul: pornești `Produs maxim = this.produse[0];` și compari `this.produse[i].pret > maxim.pret`.

Exercițiul 8 — NumaraInOferta

Adaugă `NumaraInOferta()` care întoarce **câte** produse au `inOferta == true`.

Exemplu: din 4 produse, 2 în ofertă $\rightarrow$ 2.

Contor `int` + `if (this.produse[i].inOferta)`. Filtrare + numărare, mutate în metodă.

Exercițiul 9 — ProduseInOferta întoarce o *listă nouă*

Adaugă `ProduseInOferta()` al cărei tip de retur este `List<Produs>`. Creezi o listă nouă goală, pui în ea doar produsele în ofertă, și o **întorci**.

Exemplu: din 4 produse (2 în ofertă) → întoarce o listă cu acele 2. În `Main` parcurgi rezultatul și îl afișezi.

Idee mare: o metodă poate **construi și întoarce o listă**. Tipar: `List<Produs> rezultat = new List<Produs>();` → `rezultat.Add(...)` în `if` → `return rezultat;`.

Partea 3 — Modificare (Update / Delete pe listă din clasă)

Exercițiul 10 — Sterge după nume

Adaugă `Sterge(string nume)`: găsești produsul (ca la ex. 5) și, dacă există, îl scoți din listă cu `this.produse.Remove(gasit)`. Afișezi câte au rămas.

Exemplu: coș cu 3 produse, `Sterge("bec")` → rămân 2.

Găsești întâi obiectul, apoi `Remove(obiect)` — eviți problema decalajului dintr-un `for` când ștergi după index.

Exercițiul 11 — `AplicaReducereTuturor`

Adaugă `AplicaReducereTuturor(double procent)` (metodă `void`) care parcurge lista și scade procentul din prețul **fiecărui** produs ($\text{pret} = \text{pret} - \text{pret} * \text{procent} / 100$).

Exemplu: reducere de 10 $\rightarrow$ toate prețurile scad cu 10%. Apoi `Total()` dă mai puțin.

Metodă a coșului care **modifică** fiecare obiect din listă. Verifică efectul apelând `Total()` înainte și după.

Partea 4 — Altă entitate: liste de valori într-o clasă

Exercițiul 12 — Student cu o listă de note

Extinde `Student` cu un câmp `List<double> note = new List<double>();` și metodele:

- `AdaugaNota(double nota)` — o pune în listă;
- `Media()` — întoarce media notelor (atenție la `double!`).

Exemplu: note 8, 9, 10 $\rightarrow$ `Media()` = 9.0.

Aici lista e de **valori** (`double`), nu de obiecte. Restul e la fel: parcurgi, aduni, împarți la `note.Count`. Grijă la lista goală (împărțire la 0).

Exercițiul 13 — APromovat pe baza mediei

Adaugă în `Student` metoda `APromovat() → bool`, `true` dacă `Media() >= 5`. Observă că o metodă poate chema **altă metodă** a aceleiași clase.

Exemplu: note 4, 5 → media 4.5 → `false`. Note 8, 9 → `true`.

`return this.Media() >= 5;` — o metodă `bool` care se sprijină pe o metodă de calcul.

Exercițiul 14 — Clasa Scoala cu List<Student>

Creează Scoala cu un câmp List<Student> studenti. Adaugă:

- Inscribe(Student s);
- NumarPromovati() — câți studenți au APromovat() == true;
- MediaGenerală() — media tuturor Media()-lor studenților.

Exemplu: 3 studenți cu mediile 9, 4.5, 7 → promovați 2, media generală aprox. 6.83.

Acum ai **liste imbricate**: Scoala are o listă de Student, iar fiecare Student are o listă de note. Metodele se cheamă una pe alta pe niveluri.

Exercițiul 15 (provocare) — Mini-catalog complet

Pune totul la un loc într-un `Cos` (sau `Magazin`) și rulează pe rând din `Main`, afișând ce se întâmplă la fiecare pas:

1. `Adauga` 4 produse (unele în ofertă).
2. `AfiseazaTot`.
3. `Total` și `ValoareTotala`.
4. `CelMaiScump` → afișezi numele.
5. `ProduseInOferta` → afișezi lista întoarsă.
6. `AplicaReducereTuturor(10)`, apoi din nou `Total` (mai mic).
7. `Sterge` un produs, apoi `NumaraProduse`.

Ăsta e scheletul unei aplicații reale de gestiune: **datele stau într-o listă din obiect, iar toate operațiile sunt metode**. `Main` doar dă comenzi.

Recapitulare

Vreau ca metoda să...	Tipul de retur	Tipar
adauge / ștergă / modifice în listă	<code>void</code>	<code>this.lista.Add(...)</code> <code>/ Remove(...)</code> <code>/ schimbi câmp</code>
spună câte / cât / media	<code>int</code> / <code>double</code>	parcurgi + acumulator, return
găsească un obiect	tipul obiectului (<code>Produs</code>) sau <code>null</code>	parcurgi, return
răspundă da/nu	<code>bool</code>	<code>this.lista[i];</code> return
întoarce o sub-listă filtrată	<code>List<Produs></code>	conditie; listă nouă goală → <code>Add</code> în <code>if</code> → return

Ideea centrală a materialului:

O clasă poate **deține o listă** (`List<...>` ca field). Parcurgerea cu `for (int i = 0; i < this.lista.Count; i++)` — pe care o știi deja — se mută **din Main în metode**. Main devine scurt: doar creează obiectul și îi cere lucruri. Așa e structurată orice aplicație reală (coș, catalog, școală, magazin).